

vLLM专题（十二）-推理输出（Reasoning Outputs）



大模型专题系列 专栏收录该内容

¥49.90
¥99.00

111 篇文章

已订阅

8折续

👑 您已是超级会员，正在免费阅读会员专享内容

[查看更多超级会员权益](#) >

vLLM 支持推理模型，例如 DeepSeek R1，这些模型旨在生成包含推理步骤和最终结论的输出。

推理模型在其输出中返回一个额外的 `reasoning_content` 字段，该字段包含导致最终结论的推理步骤。其他模型的输出中不存在此字段。

1、支持的模型

vLLM 目前支持以下推理模型：

- DeepSeek R1 系列（`deepseek_r1`，用于解析 `<think> ... </think>` 格式的内容）

2、快速开始

要使用推理模型，你需要在向聊天补全端点发送请求时指定 `--enable-reasoning` 和 `--reasoning-parser` 标志。`--reasoning-parser` 标志指定用于从模型输出中提取推理内容的解析器。

```
vllm serve deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B \
--enable-reasoning --reasoning-parser deepseek_r1
```

接下来，向模型发送请求，响应中应包含推理内容。

```

from openai import OpenAI

# 修改 OpenAI 的 API 密钥和 API 基础地址以使用 vLLM 的 API 服务器。
openai_api_key = "EMPTY"
openai_api_base = "http://localhost:8000/v1"

client = OpenAI(
    api_key=openai_api_key,
    base_url=openai_api_base,
)

models = client.models.list()
model = models.data[0].id

# 第一轮
messages = [{
    "role": "user", "content": "9.11 和 9.8, 哪个更大? "}]]
response = client.chat.completions.create(model=model, messages=messages)

reasoning_content = response.choices[0].message.reasoning_content
content = response.choices[0].message.content

print("推理内容:", reasoning_content)
print("最终结论:", content)

```

`reasoning_content` 字段包含导致最终结论的推理步骤，而 `content` 字段包含最终结论。

3、流式聊天补全

推理模型也支持流式聊天补全。`reasoning_content` 字段在聊天补全响应块中的 `delta` 字段中可用。

```
{
  "id": "chatcmpl-123",
  "object": "chat.completion.chunk",
  "created": 1694268190,
  "model": "deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B",
  "system_fingerprint": "fp_44709d6fcb",
  "choices": [
    {
      "index": 0,
      "delta": {
        "role": "assistant",
        "reasoning_content": "is",
      },
      "logprobs": null,
      "finish_reason": null
    }
  ]
}
```

请注意，此功能与 OpenAI Python 客户端库不兼容。你可以使用 `requests` 库来发起流式请求。

4、如何支持新的推理模型

你可以添加一个新的 `ReasoningParser`，类似于

`vllm/entrypoints/openai/reasoning_parsers/deepseek_r1_reasoning_parser.py`。

```

# 导入所需的包
from vllm.entrypoints.openai.reasoning_parsers.abs_reasoning_parsers import (
    ReasoningParser, ReasoningParserManager)
from vllm.entrypoints.openai.protocol import (ChatCompletionRequest,
                                              DeltaMessage)

# 定义一个推理解析器并注册到 vLLM
# register_module 中的名称列表可用于 --reasoning-parser
@ReasoningParserManager.register_module(["example"])
class ExampleParser(ReasoningParser):
    def __init__(self, tokenizer: AnyTokenizer):
        super().__init__(tokenizer)

    def extract_reasoning_content_streaming(
        self,
        previous_text: str,
        current_text: str,
        delta_text: str,
        previous_token_ids: Sequence[int],
        current_token_ids: Sequence[int],
        delta_token_ids: Sequence[int],
    ) -> Union[DeltaMessage, None]:
        """
        实例方法，用于从未完成的响应中提取推理内容；适用于处理推理调用和流式响应。
        必须是一个实例方法，因为它需要状态——当前的标记/差异，以及之前解析和提取的信息（参见构造函数
        ）。
        """

    def extract_reasoning_content(
        self, model_output: str, request: ChatCompletionRequest
    ) -> Tuple[Optional[str], Optional[str]]:
        """
        从完整的模型生成字符串中提取推理内容。

        用于非流式响应，即在发送给客户端之前我们已经拥有完整的模型响应。

        参数：
        model_output: str
            从中提取推理内容的模型生成字符串。

        request: ChatCompletionRequest
            用于生成 model_output 的请求对象。

        返回：
        Tuple[Optional[str], Optional[str]]
            包含推理内容和最终结论的元组。
        """

```

定义推理解析器后，你可以在向聊天补全端点发送请求时通过指定 `--reasoning-parser` 标志来使用它。

```
vllm serve <model_tag> \  
--enable-reasoning --reasoning-parser example
```

5、限制

1. 推理内容仅适用于在线服务的聊天补全端点（ `/v1/chat/completions` ）。
2. 与 `structured_outputs` 和 `tool_calling` 功能不兼容。
3. 并非所有模型都支持推理内容。请查阅模型的文档以确认其是否支持推理功能。